UNIVERSIDADE CATÓLICA DE BRASÍLIA
CIÊNCIA DA COMPUTAÇÃO

DAVID LAFAIETE LUSTOSA LISBOA
JHONY MORAES DA SILVA
JOÃO VÍTOR FERREIRA CALADO
KAUÃ LACERDA DE LIMA
RAFAEL DE SOUSA SILVA

RELATORIO DE APRESENTAÇÃO

SISTEMA GERENCIADOR DE LOJA - (SGL)

BRASÍLIA, DF
2026

DAVID LAFAIETE LUSTOSA LISBOA
JHONY MORAES DA SILVA
JOÃO VÍTOR FERREIRA CALADO
KAUÃ LACERDA DE LIMA
RAFAEL DE SOUSA SILVA

RELATORIO DE APRESENTAÇÃO

SISTEMA GERENCIADOR DE LOJA - (SGL)

Documento apresentado a Universidade Católica de Brasília, campus Taguatinga, como requisito para aprovação bimestral parcial no curso de Laboratório de Banco de Dados.
Orientador: Prof. Samuel Novais

BRASÍLIA, DF
2026

SUMÁRIO

A1 DOCUMENTO DE ESCOPO E REGRAS DE NEGÓCIO 5
A1.1 DESCRIÇÃO 5
A1.1.1 Escopo do Sistema Gerenciador de Loja (SGL) 5
A1.1.2 Funcionalidades e Arquitetura Operacional 5
A1.1.3 Diretrizes de Integridade e Regras Técnicas 6
A1.2 REGRAS 6
A2 MODELO ENTIDADE-RELACIONAMENTO CONCEITUAL 9
A3 DICIONÁRIO DE DADOS CONCEITUAL E FÍSICO 9
A3.1 TABELA PESSOA 9
A3.2 TABELA VENDEDOR 10
A3.3 TABELA CLIENTE 10
A3.4 TABELA FORNECEDOR 11
A3.5 TABELA CATEGORIA 11
A3.6 TABELA PRODUTO 12
A3.7 TABELA FORNECEDOR_PRODUTO 13
A3.8 TABELA PEDIDO 13
A3.9 TABELA ITEM_PEDIDO 14
A3.10 TABELA HISTORICO_STATUS_PEDIDO 15
A3.11 TABELA PAGAMENTO 15
A4 MODELO LÓGICO RELACIONAL 16
A4.1 ESQUEMA RELACIONAL DERIVADO DO MER 16
A4.2 JUSTIFICATIVA DE DECISÕES 17
A4.3 MAPEAMENTO DE TODO RELACIONAMENTO N:N EM TABELA ASSOCIATIVA 18
A4.3.1 Relacionamentos N:N Comuns 18
A4.3.2 Relacionamentos N:N Fracos e Itens Dependentes 18
A4.4 JUSTIFICATIVA DA ESCOLHA ENTRE CHAVE NATURAL E CHAVE SUBSTITUTA EM CADA RELAÇÃO. 18
A5 VERIFICAÇÃO DE NORMALIZAÇÃO 19
A5.1 DEPENDÊNCIAS FUNCIONAIS RELEVANTES DE CADA RELAÇÃO 19
A5.1.1 Relação: Pessoa 19
A5.1.2 Relação: Vendedor 19
A5.1.3 Relação: Cliente 20
A5.1.4 Relação: Fornecedor 20
A5.1.5 Relação: Categoria 20
A5.1.6 Relação: Produto 21
A5.1.7 Relação Associativa: Fornecedor_Produto 21
A5.1.8 Relação: Pedido 22
A5.1.9 Relação Associativa: Item_Pedido 22
A5.1.10 Relação Fraca: Historico_Status_Pedido 22
A5.1.11 Relação: Pagamento 23
A5.2 RELAÇÃO QUE TENHA SIDO DELIBERADAMENTE DESNORMALIZADA 23
A5.2.1 Relação: Item_Pedido (Atributo: subtotal) 23
A5.2.2 Relação: Pedido (Atributo: valor_total) 24
A5.2.3 Relação: Fornecedor_Produto (Atributo: id_vendedor) 24

B1 APLICAÇÃO COM INTERFACE 24
B2 CAMADA DE ACESSO A DADOS 24
B2.1 PRÁTICAS DE SEGURANÇA E CONSULTAS PARAMETRIZADAS (PREPARED STATEMENTS) 25
B2.2 DEMONSTRAÇÃO DE TESTE DE RESISTÊNCIA À INJEÇÃO DE SQL 25
B2.3 GERENCIAMENTO SEGURO DE CREDENCIAIS 26
B3 TRANSAÇÕES E CONTROLE DE CONCORRÊNCIA 26
B3.1 IMPLEMENTAÇÃO DE TRANSAÇÕES NA CAMADA DE APLICAÇÃO 26
B3.2 CONTROLE DE CONCORRÊNCIA E ISOLAMENTO (PREVENÇÃO DE CONDIÇÃO DE CORRIDA) 28
B4 OTIMIZAÇÃO DE CONSULTAS 29
B4.1 CONSULTA 1: RELATÓRIO DE VENDAS POR CLIENTE 29
B4.2 CONSULTA 2: RELATÓRIO DE PRODUTOS VENDIDOS E RECEITA 30
B4.3 CONSULTA 3: LISTAGEM DE PEDIDOS COM SUBCONSULTA DE STATUS MAIS RECENTE 30
B4.4 DISCUSSÃO HONESTA E CONCLUSÃO DOS TESTES 31
B5 SEGURANÇA E CONTROLE DE ACESSO 31
B5.1 DEFINIÇÃO E CRIAÇÃO DE PERFIS DE ACESSO (GRANT / REVOKE) 31
B5.2 RESTRIÇÃO DE ACESSO A DADOS VIA VISÃO (VIEW) 33
B5.3 TRATAMENTO E ARMAZENAMENTO DE SENHAS (HASH) 34
B5.4 TRATAMENTO DE DADOS PESSOAIS E LGPD 34
B5.4.1 Mapeamento de Dados Pessoais Identificados: 34
B5.4.2 Medidas de Proteção e Boas Práticas Adotadas: 35
B6 GATILHOS E AUDITORIA 35
B6.1 ESTRUTURA DA TABELA DE AUDITORIA 35
B6.2 IMPLEMENTAÇÃO DO GATILHO 1: AUDITORIA DE ALTERAÇÃO DE PREÇOS DE PRODUTOS 36
B6.3 IMPLEMENTAÇÃO DO GATILHO 2: AUDITORIA DE ALTERAÇÃO DE STATUS DE PEDIDOS 36
B6.4 VALIDAÇÃO E EVIDÊNCIA DE FUNCIONAMENTO 37
B7 PROJETO FÍSICO E ARMAZENAMENTO 38
B7.1 REVISÃO DOS TIPOS DE DADOS À LUZ DO DOMÍNIO REAL 38
B7.2 ESTIMATIVA DE CRESCIMENTO DAS TABELAS DE MAIOR MOVIMENTO (HORIZONTE DE 2 ANOS) 39
B7.2.1 Tabela item_pedido 39
B7.2.2 Tabela historico_status_pedido 39
B7.3 PLANO DE BACKUP, RETENÇÃO E RECUPERAÇÃO 40
B7.4 PROPOSTA DE PARTICIONAMENTO E ARQUIVAMENTO HISTÓRICO 41

A1 DOCUMENTO DE ESCOPO E REGRAS DE NEGÓCIO

A1.1 DESCRIÇÃO

A1.1.1 Escopo do Sistema Gerenciador de Loja (SGL)

O Sistema Gerenciador de Loja (SGL) tem como objetivo fornecer uma solução centralizada e robusta para o gerenciamento das operações relacionadas à comercialização, fornecimento e logística de produtos. O sistema atende comerciantes no controle de entidades fundamentais, englobando clientes, vendedores e fornecedores. Além disso, gerencia estoques, o catálogo de produtos organizados por categorias hierárquicas, e assegura a rastreabilidade completa sobre pedidos, transações de pagamentos e histórico de movimentações.

A1.1.2 Funcionalidades e Arquitetura Operacional

As operações principais do sistema estão divididas nos seguintes módulos:

- **Gestão de Pessoas, Clientes e Vendedores:** Permite o cadastro centralizado de pessoas físicas, garantindo a especialização de papéis entre Clientes (com data de cadastro controlada) e Vendedores (com credenciais de login e senha protegidas por hash), identificando claramente a autoria das vendas e o atendimento prestado.
- **Gestão de Fornecedores e Insumos:** Gerencia dados cadastrais e operacionais de empresas fornecedoras vinculadas ao vendedor responsável, mapeando condições de abastecimento, preço de custo do lote, quantidade e prazo de entrega.
- **Gestão de Categorias e Produtos:** Organiza o catálogo de produtos de forma hierárquica por meio de categorias e subcategorias (suportando autorrelacionamento), controlando preços unitários, descrições, status de comercialização e saldos de estoque.
- **Gestão de Pedidos e Itens:** Registra as vendas vinculando o cliente comprador e o vendedor responsável. O preço unitário do produto é congelado na tabela de itens no momento da transação para preservar o histórico financeiro, com cálculo automático de subtotais e desnormalização controlada do valor total do pedido.
- **Rastreabilidade e Pagamentos:** Controla cronologicamente o ciclo de vida dos pedidos através de um histórico de status auditável (PENDENTE, PAGO, FINALIZADO, CANCELADO) e processa transações financeiras com múltiplas modalidades e estados de processamento.

A1.1.3 Diretrizes de Integridade e Regras Técnicas

O sistema foi concebido para assegurar a consistência dos dados tanto a nível de banco de dados (através de restrições de chave primária, estrangeira, restrições de verificação e unicidade) quanto na camada de aplicação. A modelagem prevê mecanismos rígidos para impedir inconsistências transacionais, como a inativação lógica de registros com vínculos históricos e o bloqueio de alterações em pedidos finalizados ou cancelados.

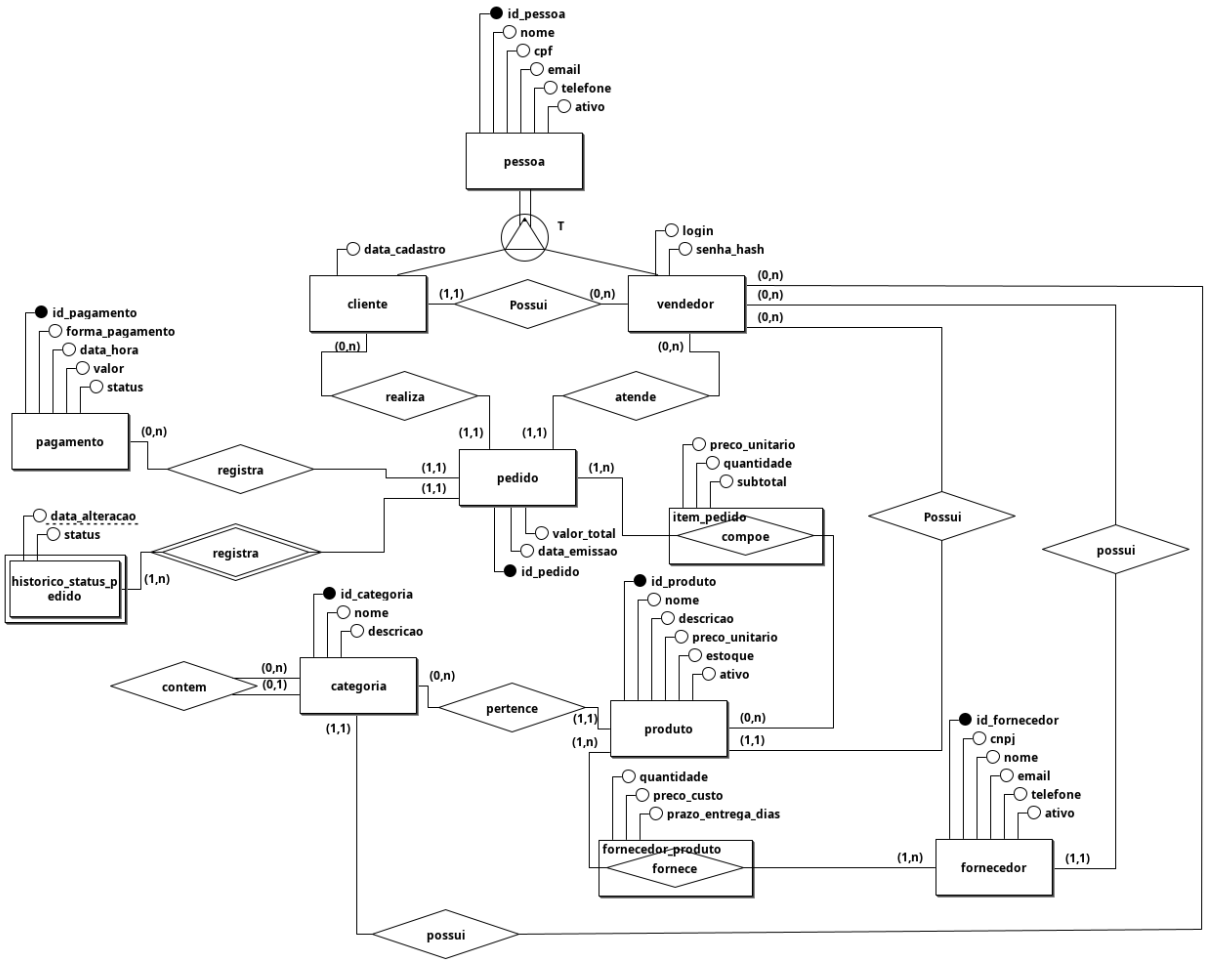
A1.2 REGRAS

- **RN01** - O CPF cadastrado para uma pessoa deve ser único no sistema e possuir exatamente 11 dígitos numéricos.
    - Forma de implementação: Banco de Dados (UNIQUE, CHECK via CHAR_LENGTH).
- **RN02** - O endereço de e-mail de uma pessoa deve ser obrigatório e único em todo o sistema.
    - Forma de implementação: Banco de Dados (NOT NULL, UNIQUE).
- **RN03** - Todo vendedor deve possuir um login único e a sua senha de acesso armazenada em formato criptografado (hash).
    - Forma de implementação: Banco de Dados (NOT NULL, UNIQUE) / Aplicação.
- **RN04** - A data de cadastro do cliente deve ser atribuída automaticamente pelo sistema no momento do registro.
    - Forma de implementação: Banco de Dados (DEFAULT CURRENT_TIMESTAMP).
- **RN05** - A especialização da entidade Pessoa em Cliente e Vendedor é Total e Sobreposta, sendo obrigatório que toda pessoa cadastrada seja vinculada como Cliente, Vendedor ou ambos simultaneamente.
    - Forma de implementação: Aplicação.
- **RN06** - O CNPJ do fornecedor deve ser único no sistema (por vendedor) e possuir a extensão exata de 14 caracteres numéricos.
    - Forma de implementação: Banco de Dados (UNIQUE, CHECK via CHAR_LENGTH).
- **RN07** - O endereço de e-mail de contato do fornecedor é obrigatório e não pode ser duplicado.
    - Forma de implementação: Banco de Dados (NOT NULL) / Aplicação.

- **RN08** - O nome de cada categoria de produto deve ser único dentro do catálogo geral de cada vendedor.
    - Forma de implementação: Banco de Dados (UNIQUE composto).
- **RN09** - Uma categoria pode se relacionar com outra categoria pai para permitir a estruturação de subcategorias hierárquicas.
    - Forma de implementação: Banco de Dados (FOREIGN KEY autorreferenciada com ON DELETE SET NULL).
- **RN10** - O preço unitário de um produto e a sua quantidade em estoque não podem assumir valores negativos.
    - Forma de implementação: Banco de Dados (CHECK).
- **RN11** - Apenas produtos com o indicador ativo marcado como verdadeiro e com estoque disponível podem ser selecionados para a criação de novos pedidos.
    - Forma de implementação: Consulta SQL / Aplicação.
- **RN12** - O relacionamento de fornecimento entre fornecedor e produto deve registrar quantidade maior que zero, preço de custo não-negativo e prazo de entrega superior a zero dias.
    - Forma de implementação: Banco de Dados (CHECK).
- **RN13** - Todo pedido deve estar obrigatoriamente associado a um Cliente existente e a um Vendedor responsável.
    - Forma de implementação: Banco de Dados (FOREIGN KEY, NOT NULL).
- **RN14** - Um pedido deve possuir pelo menos um item vinculado para ser processado e salvo no sistema.
    - Forma de implementação: Aplicação.
- **RN15** - A quantidade de cada item em um pedido deve ser estritamente maior do que zero unidades.
    - Forma de implementação: Banco de Dados (CHECK).
- **RN16** - O preço unitário do produto deve ser copiado e congelado na tabela de itens do pedido no momento do cadastro para evitar alteração retroativa em vendas antigas.
    - Forma de implementação: Banco de Dados / Aplicação.
- **RN17** - O subtotal do item é obtido pela multiplicação da quantidade pelo preço unitário praticado na transação, devendo ser não-negativo e validado de forma consistente.
    - Forma de implementação: Banco de Dados (CHECK com validação de cálculo) / Aplicação.

- **RN18** - O valor total do pedido deve armazenar o somatório dos subtotais dos itens compostos como estratégia de desnormalização pontual para leitura otimizada.
    - Forma de implementação: Banco de Dados (CHECK) / Aplicação.
- **RN19** - A confirmação de inclusão de um item no pedido deve abater automaticamente a quantidade vendida do estoque atual do produto.
    - Forma de implementação: Aplicação.
- **RN20** - Todo pedido recém-criado deve obrigatoriamente registrar um evento inicial no seu histórico com o status 'PENDENTE'.
    - Forma de implementação: Banco de Dados (DEFAULT) / Aplicação.
- **RN21** - Os valores aceitos para atualização do status no histórico do pedido são estritamente restritos a: 'PENDENTE', 'PAGO', 'FINALIZADO', 'CANCELADO'.
    - Forma de implementação: Banco de Dados (CHECK com operador IN).
- **RN22** - O histórico de status de cada pedido é identificado por chave primária composta (id_pedido, data_alteracao) para garantir rastreabilidade cronológica auditável.
    - Forma de implementação: Banco de Dados (PRIMARY KEY composta, FOREIGN KEY).
- **RN23** - Pedidos que atingirem o status 'FINALIZADO' ou 'CANCELADO' ficam bloqueados contra alteração de itens, quantidades ou valores associados.
    - Forma de implementação: Aplicação.
- **RN24** - O valor informado para qualquer transação de pagamento deve ser estritamente maior que zero.
    - Forma de implementação: Banco de Dados (CHECK).
- **RN25** - As formas de pagamento aceitas pelo sistema são restritas a: 'DINHEIRO', 'CARTAO_CREDITO', 'CARTAO_DEBITO', 'PIX', 'BOLETO'.
    - Forma de implementação: Banco de Dados (CHECK com operador IN).
- **RN26** - O estado de processamento da transação de pagamento é restrito aos valores: 'PROCESSANDO', 'APROVADO', 'RECUSADO', 'ESTORNADO'.
    - Forma de implementação: Banco de Dados (CHECK com operador IN).
- **RN27** - A remoção de pessoas, produtos ou fornecedores que possuam vínculos transacionais históricos deve ser realizada via inativação lógica através do atributo booleano ativo.
    - Forma de implementação: Consulta SQL / Aplicação.

A2 MODELO ENTIDADE-RELACIONAMENTO CONCEITUAL



A3 DICIONÁRIO DE DADOS CONCEITUAL E FÍSICO

A3.1 TABELA PESSOA

Descrição: Armazena as informações cadastrais básicas das pessoas físicas do sistema. Atua como superclasse para Vendedor e Cliente na especialização total e sobreposta.

Regras de Negócio Atendidas (RNs): RN01 (CPF único e 11 dígitos), RN02 (E-mail obrigatório e único), RN05 (Especialização Total/Sobreposta em Cliente e Vendedor), RN27 (Inativação lógica via atributo ativo).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PESSOA	Identificador único da pessoa.	Inteiro positivo (> 0)	Obrigatório	Chave Primária (PK)	-
NOME	Nome completo da pessoa.	Texto (até 100 char)	Obrigatório	Nenhuma	-
CPF	Cadastro de Pessoa Física.	Texto fixo (11 char)	Obrigatório	Único (UNIQUE),	RN01

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
EMAIL	Endereço de email.	Texto (até 120 char)	Obrigatório	CHECK (tam=11) Único (UNIQUE)	RN02
TELEFONE	Número de telefone para contato.	Texto (até 20 char)	Opcional	Nenhuma	-
ATIVO	Indicador de status ativo/inativo.	Booleano (TRUE/FALSE)	Obrigatório	Padrão TRUE	RN27

A3.2 TABELA VENDEDOR

Descrição: Especialização da entidade Pessoa referente aos vendedores que operam no sistema com credenciais de acesso.

Regras de Negócio Atendidas (RNs): RN03 (Login único e senha em hash), RN05 (Especialização de Pessoa), RN27 (Inativação lógica).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PESSOA	Identificador único do vendedor (herdado de pessoa).	Inteiro positivo (> 0)	Obrigatório	PK, FK referenciando pessoa(id_pessoa)	RN05
LOGIN	Nome de usuário para autenticação.	Texto (até 50 char)	Obrigatório	Único (UNIQUE)	RN03
SENHA_HASH	Senha de acesso criptografada em hash.	Texto (até 255 char)	Obrigatório	Nenhuma	RN03

A3.3 TABELA CLIENTE

Descrição: Especialização da entidade Pessoa referente aos clientes cadastrados pelos vendedores.

Regras de Negócio Atendidas (RNs): RN04 (Data de cadastro automática), RN05 (Especialização de Pessoa), RN27 (Inativação lógica).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PESSOA	Identificador único do cliente (herdado de pessoa).	Inteiro positivo (> 0)	Obrigatório	PK, FK referenciando pessoa(id_pessoa)	RN05
ID_VENDEDOR	Identificador do vendedor	Inteiro positivo (> 0)	Obrigatório	FK referenciando vendedor(id_pessoa)	-

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
	responsável pelo cadastro.				
DATA_CADASTRO	Data e hora do cadastro do cliente.	Data/Hora (DATETIME)	Obrigatório	Padrão CURRENT_TIMESTAMP	RN04

A3.4 TABELA FORNECEDOR

Descrição: Armazena informações das empresas fornecedoras vinculadas a um vendedor específico.

Regras de Negócio Atendidas (RNs): RN06 (CNPJ de 14 dígitos único por vendedor), RN07 (E-mail obrigatório), RN27 (Inativação lógica).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RNRELACIONADA
ID_FORNECEDOR	Identificador único do fornecedor.	Inteiro positivo (> 0)	Obrigatório	Chave Primária (PK)	-
ID_VENDEDOR	Identificador do vendedor proprietário.	Inteiro positivo (> 0)	Obrigatório	FK referenciando vendedor(id_pess oa)	-
CNPJ	CNPJ do fornecedor.	Texto fixo (14 char)	Obrigatório	UNIQUE(cnpj, id_vendedor), CHECK(tam=14)	RN06
NOME	Razão social ou nome fantasia.	Texto (até 100 char)	Obrigatório	Nenhuma	-
EMAIL	Endereço de e-mail de contato.	Texto (até 120 char)	Obrigatório	Nenhuma	RN07
TELEFONE	Telefone de contato.	Texto (até 20 char)	Opcional	Nenhuma	-
ATIVO	Indicador de status ativo/inativo.	Booleano (TRUE/FALSE)	Obrigatório	Padrão TRUE	RN27

A3.5 TABELA CATEGORIA

Descrição: Representa a estrutura de categorias do catálogo de produtos de cada vendedor, suportando hierarquia entre categorias e subcategorias.

Regras de Negócio Atendidas (RNs): RN08 (Nome único por vendedor), RN09 (Autorrelacionamento de categorias com ON DELETE SET NULL).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_CATEGORIA	Identificador único da categoria.	Inteiro positivo (> 0)	Obrigatório	Chave Primária (PK)	-
ID_VENDEDOR	Identificador do vendedor proprietário.	Inteiro positivo (> 0)	Obrigatório	FK referenciando vendedor(id_pessoa)	-
NOME	Nome da categoria.	Texto (até 100 char)	Obrigatório	UNIQUE(nome, id_vendedor)	RN08
DESCRICAO	Descrição detalhada da categoria.	Texto (até 255 char)	Opcional	Nenhuma	-
ID_CATEGORIA_PAI	Identificador da categoria pai (para subcategorias).	Inteiro positivo (> 0)	Opcional	FK referenciando categoria(id_categoria)	RN09

A3.6 TABELA PRODUTO

Descrição: Armazena os dados dos produtos comercializados por cada vendedor.

Regras de Negócio Atendidas (RNs): RN10 (Preço e estoque não-negativos), RN11 (Seleção de produtos ativos com estoque), RN27 (Inativação lógica).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PRODUTO	Identificador único do produto.	Inteiro positivo (> 0)	Obrigatório	Chave Primária (PK)	-
ID_VENDEDOR	Identificador do vendedor proprietário.	Inteiro positivo (> 0)	Obrigatório	FK referenciando vendedor(id_pessoa)	-
ID_CATEGORIA	Identificador da categoria do produto.	Inteiro positivo (> 0)	Obrigatório	FK referenciando categoria(id_categoria, id_vendedor)	-
NOME	Nome do produto.	Texto (até 100 char)	Obrigatório	Nenhuma	-
DESCRICAO	Descrição do produto.	Texto (até 255 char)	Opcional	Nenhuma	-
PRECO_UNITARIO	Preço unitário de venda.	Decimal (10,2) >= 0	Obrigatório	CHECK(preco_unitario >= 0)	RN10

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ESTOQUE	Quantidade atual em estoque.	Inteiro >= 0	Obrigatório	Padrão 0, CHECK(estoque >= 0)	RN10, RN11
ATIVO	Indicador de status ativo/inativo	Booleano (TRUE/FALSE)	Obrigatório	Padrão TRUE	RN11, RN27

A3.7 TABELA FORNECEDOR_PRODUTO

Descrição: Tabela associativa entre Fornecedor e Produto para registrar condições de fornecimento.

Regras de Negócio Atendidas (RNs): RN12 (Quantidade > 0, preço de custo >= 0, prazo de entrega > 0).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_FORNECEDOR	Identificador do fornecedor.	Inteiro positivo (> 0)	Obrigatório	PK Composta, FK referenciando fornecedor(id_fornecedor, id_vendedor)	-
ID_PRODUTO	Identificador do produto fornecido.	Inteiro positivo (> 0)	Obrigatório	PK Composta, FK referenciando produto(id_produto, id_vendedor)	-
ID_VENDEDOR	Identificador do vendedor proprietário.	Inteiro positivo (> 0)	Obrigatório	FK referenciando vendedor(id_pessoa)	-
QUANTIDADE	Quantidade fornecida no lote.	Inteiro positivo (> 0)	Obrigatório	CHECK(quantidade > 0)	RN12
PRECO_CUSTO	Preço de custo unitário do fornecedor.	Decimal (10,2) >= 0	Obrigatório	CHECK(preco_custo >= 0)	RN12
PRAZO_ENTREGA_DIAS	Prazo de entrega em dias.	Inteiro positivo (> 0)	Obrigatório	Padrão 1, CHECK(prazo_entrega_dias > 0)	RN12

A3.8 TABELA PEDIDO

Descrição: Registra as vendas efetuadas pelos vendedores para os seus clientes.

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PEDIDO	Identificador único do pedido.	Inteiro positivo (> 0)	Obrigatório	Chave Primária (PK)	-
ID_CLIENTE	Identificador do cliente comprador.	Inteiro positivo (> 0)	Obrigatório	FK referenciando cliente(id_pessoa, id_vendedor)	RN13
ID_VENDEDOR	Identificador do vendedor responsável.	Inteiro positivo (> 0)	Obrigatório	FK referenciando vendedor(id_pessoa)	RN13
DATA_EMISSAO	Data e hora de emissão do pedido.	Data/Hora (DATETIME)	Obrigatório	Padrão CURRENT_TIMESTAMP	-
VALOR_TOTAL	Valor total do pedido.	Decimal (10,2) >= 0	Obrigatório	Padrão 0.00, CHECK(valor_total >= 0)	RN18

A3.9 TABELA ITEM_PEDIDO

Descrição: Tabela associativa fraca que detalha os produtos contidos em cada pedido.

Regras de Negócio Atendidas (RNs): RN15 (Quantidade > 0), RN16 (Preço congelado no momento da venda), RN17 (Subtotal = quantidade * preço unitário), RN19 (Abatimento de estoque ao incluir item).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PEDIDO	Identificador do pedido.	Inteiro positivo (> 0)	Obrigatório	PK Composta, FK referenciando pedido(id_pedido)	-
ID_PRODUTO	Identificador do produto vendido.	Inteiro positivo (> 0)	Obrigatório	PK Composta, FK referenciando produto(id_produto)	-
QUANTIDADE	Quantidade do produto no item.	Inteiro positivo (> 0)	Obrigatório	CHECK(quantidade > 0)	RN15, RN19
PRECO_UNITARIO	Preço unitário congelado na venda.	Decimal (10,2) >= 0	Obrigatório	CHECK(preco_unitario >= 0)	RN16
SUBTOTAL	Valor subtotal do item.	Decimal (10,2) >= 0	Obrigatório	CHECK(subtotal = quantidade * preco_unitario)	RN17

A3.10 TABELA HISTORICO_STATUS_PEDIDO

Descrição: Tabela fraca que armazena o histórico cronológico de alteração do status de cada pedido.

Regras de Negócio Atendidas (RNs): RN20 (Evento inicial obrigatoriamente PENDENTE), RN21 (Status restritos a PENDENTE, PAGO, FINALIZADO, CANCELADO), RN22 (PK Composta id_pedido, data_alteracao).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PEDIDO	Identificador do pedido.	Inteiro positivo (> 0)	Obrigatório	PK Composta, FK referenciando pedido(id_pedido)	RN22
DATA_ALTERACAO	Data e hora do registro da alteração.	Data/Hora (DATETIME)	Obrigatório	PK Composta, Padrão CURRENT_TIMESTAMP	RN22
STATUS	Status do pedido.	Texto (até 20 char)	Obrigatório	CHECK(status IN ('PENDENTE','PAGO','FINALIZADO','CANCELADO'))	RN20, RN21

A3.11 TABELA PAGAMENTO

Descrição: Armazena os dados das transações financeiras de pagamento realizadas para os pedidos.

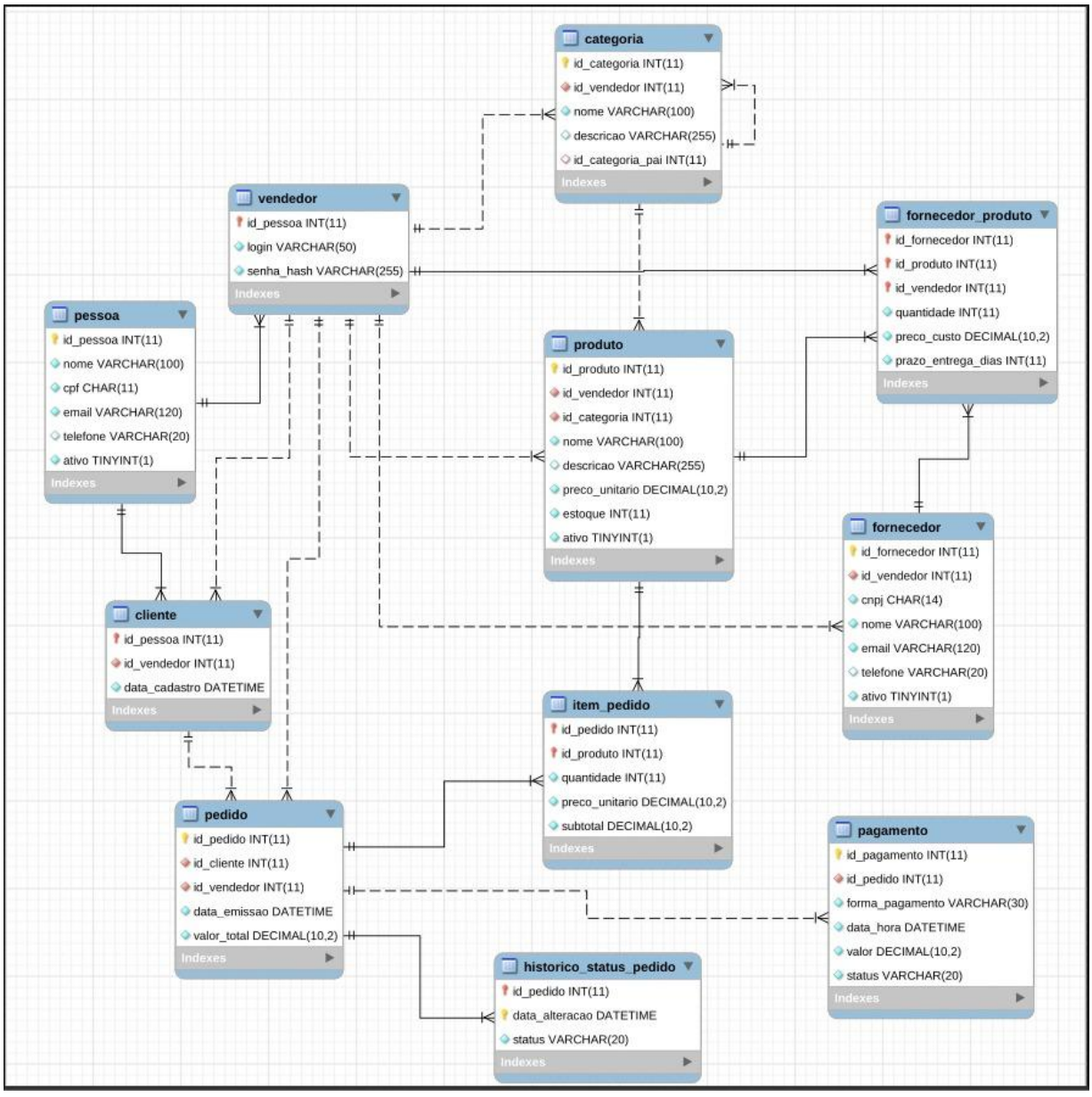
Regras de Negócio Atendidas (RNs): RN24 (Valor estritamente > 0), RN25 (Formas de pagamento válidas), RN26 (Status de processamento válidos).

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
ID_PAGAMENTO	Identificador único do pagamento.	Inteiro positivo (> 0)	Obrigatório	Chave Primária (PK)	-
ID_PEDIDO	Identificador do pedido pago.	Inteiro positivo (> 0)	Obrigatório	FK referenciando pedido(id_pedido)	-
FORMA_PAGAMENTO	Forma de pagamento utilizada.	Texto (até 30 char)	Obrigatório	CHECK(DINHEIRO, CARTAO_CREDITO, CARTAO_DEBITO, PIX, BOLETO)	RN25
DATA_HORA	Data e hora de efetivação do	Data/Hora (DATETIME)	Obrigatório	Padrão CURRENT_TIMESTAMP	-

NOME DO ATRIBUTO	DESCRIÇÃO	TIPO DE DADO	OBRIGATORIEDADE	CHAVE / RESTRIÇÃO	RN RELACIONADA
	pagamento.				
VALOR	Valor monetário pago.	Decimal (10,2) > 0	Obrigatório	CHECK(valor > 0)	RN24
STATUS	Status de processamento do pagamento.	Texto (até 20 char)	Obrigatório	Padrão 'PROCESSANDO', CHECK(PROCESSANDO, APROVADO, RECUSADO, ESTORNADO)	RN26

A4 MODELO LÓGICO RELACIONAL

A4.1 ESQUEMA RELACIONAL DERIVADO DO MER



- **pessoa** (id_pessoa [PK], nome, cpf [UQ], email [UQ], telefone, ativo)
- **vendedor** (id_pessoa [PK, FK -> pessoa.id_pessoa], login [UQ], senha_hash)
- **cliente** (id_pessoa [PK, FK -> pessoa.id_pessoa], id_vendedor [FK -> vendedor.id_pessoa], data_cadastro)
- **fornecedor** (id_fornecedor [PK], id_vendedor [FK -> vendedor.id_pessoa], cnpj [UQ], nome, email, telefone, ativo)
- **categoria** (id_categoria [PK], id_vendedor [FK -> vendedor.id_pessoa], nome, descricao, id_categoria_pai [FK -> categoria.id_categoria])
- **produto** (id_produto [PK], id_vendedor [FK -> vendedor.id_pessoa], id_categoria [FK ➔ categoria.id_categoria], nome, descricao, preco_unitario, estoque, ativo)
- **fornecedor_produto** (id_fornecedor [PK, FK -> fornecedor.id_fornecedor], id_produto [PK, FK -> produto.id_produto], id_vendedor [FK -> vendedor.id_pessoa], quantidade, preco_custo, prazo_entrega_dias)
- **pedido** (id_pedido [PK], id_cliente [FK -> cliente.id_pessoa], id_vendedor [FK -> vendedor.id_pessoa], data_emissao, valor_total)
- **item_pedido** (id_pedido [PK, FK -> pedido.id_pedido], id_produto [PK, FK -> produto.id_produto], quantidade, preco_unitario, subtotal)
- **historico_status_pedido** (id_pedido [PK, FK -> pedido.id_pedido], data_alteracao
- [PK], status)
- **pagamento** (id_pagamento [PK], id_pedido [FK -> pedido.id_pedido], forma_pagamento, data_hora, valor, status)

A4.2 JUSTIFICATIVA DE DECISÕES

- **Mapeamento da Especialização de Pessoa (pessoa, cliente, vendedor):** Utilizou-se a estratégia de Tabela por Classe (Herança). A tabela pai pessoa centraliza os dados cadastrais genéricos e garante a unicidade global de cpf e email. As tabelas filhas armazenam os atributos específicos de cada papel, evitando o desperdício de espaço e a proliferação de valores nulos (NULL) em campos sensíveis de segurança (como senha_hash).
- **Arquitetura Multivendedor (id_vendedor):** Para atender às regras de negócio de isolamento de dados do sistema, inseriu-se a coluna id_vendedor nas entidades operacionais (cliente, fornecedor, categoria, produto). Isso garante que cada vendedor gerencie exclusivamente seu próprio ambiente. Por meio de restrições de unicidade

compostas, assegura-se que cada entidade operacional seja vinculada a um vendedor, dessa forma o banco irá verificar se por exemplo o produto “mouse” é único dentro do ambiente do vendedor logado, e não no ambiente de todos os vendedores, assim impedindo o cruzamento de dados entre vendedores.

A4.3 MAPEAMENTO DE TODO RELACIONAMENTO N:N EM TABELA ASSOCIATIVA

A4.3.1 Relacionamentos N:N Comuns

- **Fornecedor e Produto (fornecedor_produto):** O relacionamento N:N entre fornecedor e produto foi convertido na tabela associativa fornecedor_produto. Sua chave primária é composta por (id_fornecedor, id_produto, id_vendedor), abrigando atributos transacionais essenciais para a gestão de compras, como quantidade, preco_custo e prazo_entrega_dias.

A4.3.2 Relacionamentos N:N Fracos e Itens Dependentes

- **Pedido e Produto (item_pedido):** Representa o relacionamento N:N entre pedido e produto mapeado como tabela associativa transacional. Possui chave primária composta (id_pedido, id_produto) e cumpre o papel fundamental de guardar a quantidade e congelar o preço de venda (preco_unitario e subtotal) no momento exato da transação, protegendo o histórico financeiro contra alterações futuras no cadastro do produto.
- **Histórico de Status do Pedido (historico_status_pedido):** Entidade fraca dependente da existência de um pedido. Utiliza chave primária composta por (id_pedido, data_alteracao) para registrar de forma auditável e cronológica todas as mudanças de estado do pedido.

A4.4 JUSTIFICATIVA DA ESCOLHA ENTRE CHAVE NATURAL E CHAVE SUBSTITUTA EM CADA RELAÇÃO

- **Adoção de Chaves Substitutas Numéricas (id_* com AUTO_INCREMENT):** Foram adotadas nas principais entidades do sistema (pessoa, fornecedor, categoria, produto, pedido, pagamento). A escolha otimiza o desempenho de operações de junção (JOIN), uma vez que chaves inteiras consomem menos memória e processamento do que chaves baseadas em textos longos. Além disso, isola o modelo lógico de eventuais correções de dados do mundo real (como alterações de CPF ou CNPJ).
- **Preservação e Restrição das Chaves Naturais:** Atributos como cpf, cnpj, email e login não foram descartados; eles foram mantidos como chaves candidatas e protegidos por

restrições do tipo UNIQUE e CHECK (por exemplo, validação de tamanho de caracteres). Isso assegura o rigor das regras de negócio sem penalizar a performance das chaves estrangeiras.

A5 VERIFICAÇÃO DE NORMALIZAÇÃO

O presente documento demonstra que o esquema lógico do banco de dados encontra-se estruturado em conformidade com as formas normais da teoria de banco de dados relacional (atingindo majoritariamente a Terceira Forma Normal - 3FN e a Forma Normal de BoyceCodd-FNBC), detalhando as dependências funcionais (DFs) de cada relação e as decisões de desnormalização pontual adotadas por motivos de desempenho.

A5.1 DEPENDÊNCIAS FUNCIONAIS RELEVANTES DE CADA RELAÇÃO

A5.1.1 Relação: Pessoa

- **Atributos:** id_pessoa (PK), nome, cpf (UQ), email (UQ), telefone, ativo.
- **Dependências Funcionais:**
  - DF1: id_pessoa -> {nome, cpf, email, telefone, ativo}
  - DF2: cpf -> {id_pessoa, nome, email, telefone, ativo}
  - DF3: email -> {id_pessoa, nome, cpf, telefone, ativo}
- **Análise de Normalização:**
  - 1FN: Todos os atributos contêm apenas valores atômicos e escalares.
  - 2FN: Como a chave primária (id_pessoa) é simples, não existem dependências parciais.
  - 3FN: Não há dependências transitivas entre atributos não-chave.
  - FNBC: Todos os determinantes (id_pessoa, cpf, email) são chaves candidatas ou superchaves. A relação alcança a FNBC.

A5.1.2 Relação: Vendedor

- **Atributos: id_pessoa (PK/FK), login (UQ), senha_hash.**
- **Dependências Funcionais:**
  - DF1: id_pessoa -> {login, senha_hash}
  - DF2: login -> {id_pessoa, senha_hash}
- **Análise de Normalização:**

1FN, 2FN e 3FN: A chave primária é simples (id_pessoa) e os atributos nãochave dependem diretamente dela sem transitividade.

FNBC: Os determinantes (id_pessoa e login) são superchaves. Alcança a FNBC.

A5.1.3 Relação: Cliente

- **Atributos:** id_pessoa (PK/FK), id_vendedor (FK), data_cadastro.
- **Dependências Funcionais:**
  - DF1: id_pessoa -> {id_vendedor, data_cadastro}
- **Análise de Normalização:**
  - 1FN, 2FN e 3FN: A chave primária é simples (id_pessoa). A chave estrangeira id_vendedor e o atributo data_cadastro dependem funcionalmente da chave primária. Não há dependências transitivas entre atributos não-chave.
  - FNBC: O determinante id_pessoa é a chave primária (superchave). Alcança a FNBC.

A5.1.4 Relação: Fornecedor

- **Atributos:** id_fornecedor (PK), id_vendedor (FK), cnpj, nome, email, telefone, ativo. (Nota: possui chave única composta (cnpj, id_vendedor) e (id_fornecedor, id_vendedor)).
- **Dependências Funcionais:**
  - DF1: id_fornecedor -> {id_vendedor, cnpj, nome, email, telefone, ativo} DF2: {cnpj, id_vendedor} -> {id_fornecedor, nome, email, telefone, ativo}
- **Análise de Normalização:**
  - 1FN: Atributos atômicos e escalares.
  - 2FN: A chave primária id_fornecedor é simples, eliminando dependências parciais.
  - 3FN: Não existem dependências transitivas entre atributos não-chave.
  - FNBC: Os determinantes (id_fornecedor e o par cnpj, id_vendedor) são superchaves. Alcança a FNBC.

A5.1.5 Relação: Categoria

- **Atributos:** id_categoria (PK), id_vendedor (FK), nome, descricao, id_categoria_pai (FK). (Possui chave única composta (nome, id_vendedor) e (id_categoria, id_vendedor)).
- **Dependências Funcionais:**

  - DF1: id_categoria -> {id_vendedor, nome, descricao, id_categoria_pai} DF2: {nome, id_vendedor} -> {id_categoria, descricao, id_categoria_pai}
- **Análise de Normalização:**
  - 1FN e 2FN: Atributos atômicos e chave primária simples.
  - 3FN: O autorrelacionamento (id_categoria_pai) referencia a própria tabela sem gerar dependência transitiva entre atributos não-chave.
  - FNBC: Os determinantes são superchaves da relação. Alcança a FNBC.

A5.1.6 Relação: Produto

- **Atributos:** id_produto (PK), id_vendedor (FK), id_categoria (FK), nome, descricao, preco_unitario, estoque, ativo.
- **Dependências Funcionais:**
  - DF1: id_produto -> {id_vendedor, id_categoria, nome, descricao, preco_unitario, estoque, ativo}
- **Análise de Normalização:**
  - 1FN, 2FN e 3FN: Chave primária simples (id_produto). As chaves estrangeiras atuam como referências e nenhum atributo não-chave determina outro.
  - FNBC: O determinante id_produto é a chave primária. Alcança a FNBC.

A5.1.7 Relação Associativa: Fornecedor_Produto

- **Atributos:** id_fornecedor (PK/FK), id_produto (PK/FK), id_vendedor (FK), quantidade, preco_custo, prazo_entrega_dias.
- **Dependências Funcionais:**
  - DF1: id_fornecedor -> id_vendedor
  - DF2: {id_fornecedor, id_produto} -> {id_vendedor, quantidade, preco_custo, prazo_entrega_dias}
  - DF3: id_produto -> id_vendedor
- **Análise de Normalização:**
  - 1FN: Atende, atributos atomicos e escalares.
  - 2FN: Viola, a chave primária é composta por (id_fornecedor, id_produto). No entanto, o atributo id_vendedor depende funcionalmente apenas de id_fornecedor, sendo uma parte da chave primária oque cria uma dependência parcial.
  - 3FN: violada, pela falha no atendidmento da 2FN.

- FNBC: Não atende. A dependência funcional id_fornecedor → id_vendedor viola a FNBC, pois id_fornecedor não é superchave da relação.

A5.1.8 Relação: Pedido

- **Atributos:** id_pedido (PK), id_cliente (FK), id_vendedor (FK), data_emissao, valor_total.
- **Dependências Funcionais:**
  - DF1: id_pedido -> {id_cliente, id_vendedor, data_emissao, valor_total}
- **Análise de Normalização:**
  - 1FN e 2FN: Chave primária simples (id_pedido), sem dependências parciais.
  - 3FN: O atributo valor_total é computado a partir dos itens do pedido. No contexto estrito da tabela, a chave primária determina todos os campos, mas a presença de um valor agregado caracteriza uma desnormalização controlada para otimização de leitura, mantendo-se funcionalmente operável em nível de 3FN.
  - FNBC: O determinante id_pedido é uma superchave.

A5.1.9 Relação Associativa: Item_Pedido

- **Atributos:** id_pedido (PK/FK), id_produto (PK/FK), quantidade, preco_unitario, subtotal.
- **Dependências Funcionais:**
  - DF1: {id_pedido, id_produto} -> {quantidade, preco_unitario, subtotal}
  - DF2: {quantidade, preco_unitario} -> subtotal
- **Análise de Normalização:**
  - 1FN e 2FN: Atributos atômicos e dependência total da chave primária composta.
  - 3FN: A presença da DF2 ({quantidade, preco_unitario} -> subtotal) configura uma dependência funcional entre atributos não-chave, violando estritamente a 3FN. Esta violação é intencional e controlada por regra de restrição no banco (ck_item_subtotal_calculo).

A5.1.10 Relação Fraca: Historico_Status_Pedido

- **Atributos:** id_pedido (PK/FK), data_alteracao (PK), status.
- **Dependências Funcionais:**
  - DF1: {id_pedido, data_alteracao} -> status
- **Análise de Normalização:**

- 1FN, 2FN e 3FN: Atributos atômicos, chave composta e ausência de dependências transitivas.
- FNBC: O determinante é a chave primária composta (superchave). Alcança a FNBC.

A5.1.11 Relação: Pagamento

- **Atributos:** id_pagamento (PK), id_pedido (FK), forma_pagamento, data_hora, valor, status.
- **Dependências Funcionais:**
  - DF1: id_pagamento -> {id_pedido, forma_pagamento, data_hora, valor, status}
- **Análise de Normalização:**
  - 1FN, 2FN e 3FN: Chave primária simples (id_pagamento) e ausência de dependências transitivas.
  - FNBC: O determinante é a chave primária (superchave). Alcança a FNBC.

A5.2 RELAÇÃO QUE TENHA SIDO DELIBERADAMENTE DESNORMALIZADA

Para otimizar o desempenho de consultas frequentes e evitar operações custosas de agregação e processamento em tempo de execução (especialmente em relatórios e listagens de vendas), o modelo adota duas desnormalizações pontuais e controladas:

A5.2.1 Relação: Item_Pedido (Atributo: subtotal)

- **Violação Teórica:** O atributo subtotal é derivado diretamente do produto aritmético entre quantidade e preco_unitario (subtotal = quantidade * preco_unitario), o que viola a estipulação da Terceira Forma Normal por introduzir dependência entre atributos nãochave.
- **Justificativa de Desempenho:** Armazenar o subtotal fisicamente evita a necessidade de recalcular a expressão matemática a cada acesso ou junção de itens do pedido em telas de detalhamento de vendas e faturas, garantindo alta performance de leitura e estabilidade histórica caso o preço unitário do produto venha a sofrer alterações futuras. A integridade é garantida via constraint de validação no DDL (ck_item_subtotal_calculo).

A5.2.2 Relação: Pedido (Atributo: valor_total)

- **Violação Teórica:** O atributo valor_total representa o somatório dos subtotais dos itens associados na tabela item_pedido, constituindo um dado agregado derivado.
- **Justificativa de Desempenho:** Calcular o valor total de um pedido em tempo de execução através de funções de agregação (SUM) sobre milhares de registros exigiria leituras recorrentes e uso intensivo de processamento e memória RAM a cada listagem do histórico de pedidos. O armazenamento físico do valor_total assegura acesso imediato (tempo constante) nas consultas do sistema, mantendo a consistência transacional gerenciada pela aplicação e por gatilhos/transações.

A5.2.3 Relação: Fornecedor_Produto (Atributo: id_vendedor)

- **Violação Teórica:** O atributo id_vendedor apresenta dependência funcional parcial em relação à chave primária composta {id_fornecedor, id_produto}, uma vez que id_fornecedor → id_vendedor.
- **Justificativa de Integridade e Desempenho:** A manutenção física de id_vendedor na tabela associativa permite validar, por meio de chaves estrangeiras compostas, se o fornecedor e o produto relacionados pertencem ao mesmo vendedor. As restrições fk_fp_fornecedor e fk_fp_produto garantem essa correspondência com as respectivas tabelas de origem, enquanto fk_fp_vendedor reforça a integridade referencial do vendedor. Essa estrutura evita associações inconsistentes entre fornecedores, produtos e vendedores, preservando o isolamento dos dados de cada vendedor no sistema.

B1 APLICAÇÃO COM INTERFACE

Link do repositório com a aplicação completa:
https://github.com/jhony0413/Laboratorio_de_banco_de_dados-projeto

B2 CAMADA DE ACESSO A DADOS

A camada de acesso a dados do Sistema Gerenciador de Loja (SGL) foi desenvolvida em Node.js utilizando o driver oficial e performático mysql2, estruturando rotas RESTful robustas que garantem a comunicação segura, transacional e isolada com o banco de dados relacional MySQL.

B2.1 PRÁTICAS DE SEGURANÇA E CONSULTAS PARAMETRIZADAS (PREPARED STATEMENTS)

Para prevenir vulnerabilidades críticas como a Injeção de SQL (SQL Injection), nenhuma consulta SQL do sistema é montada por meio de concatenação ou interpolação direta de strings com dados enviados pelo usuário. Toda a interação com parâmetros de entrada (como formulários de login, cadastros e filtros de busca) utiliza consultas parametrizadas (prepared statements) através de marcadores de posição (?).

Abaixo, destaca-se um trecho representativo da camada de acesso a dados implementada no servidor da aplicação (server.js), demonstrando o uso de parâmetros na rota de autenticação de vendedores:

```
// Exemplo de rota protegida com consulta parametrizada em server.js
app.post('/api/login', (req, res) => {
    const { login, senha } = req.body;
    const query = `
        SELECT v.id_pessoa AS id_vendedor, p.nome, v.login
        FROM vendedor v
        JOIN pessoa p ON v.id_pessoa = p.id_pessoa
        WHERE v.login = ? AND v.senha_hash = ? AND p.ativo = TRUE
    `;
    // O driver trata o array de parâmetros estritamente como dados literais
    db.query(query, [login, senha], (err, results) => {
        if (err) {
            console.error('Erro no login:', err);
            return res.status(500).json({ error: 'Erro no servidor' });
        }

        if (results.length === 0) {
            return res.status(401).json({ error: 'Usuário ou senha inválidos!' });
        }

        const vendedor = results[0];
        res.json({
            message: 'Login realizado com sucesso!',
            vendedor: {
                id_vendedor: vendedor.id_vendedor,
                nome: vendedor.nome,
                login: vendedor.login
            }
        });
    });
});
```

B2.2 DEMONSTRAÇÃO DE TESTE DE RESISTÊNCIA À INJEÇÃO DE SQL

Para comprovar a eficácia das consultas parametrizadas adotadas no SGL, foi realizado um teste prático de simulação de ataque de Injeção de SQL na rota de login e nas buscas do sistema.

Entrada usada (Payload malicioso injetado):

```
' OR '1'='1
```

- **Comportamento esperado em um cenário vulnerável:** Se a query fosse montada por concatenação de strings (ex: SELECT * FROM vendedor WHERE login = '${login}'), o payload alteraria a estrutura lógica do comando SQL original, fazendo com que a condição do WHERE avaliasse para verdadeira (TRUE) independentemente da senha, permitindo o bypass de autenticação e o acesso indevido à conta do primeiro usuário administrador.
- **Comportamento real observado no SGL (Protegido):** Como o driver do MySQL utiliza prepared statements, o SGBD tratou a string inteira ' OR '1'='1 de forma estritamente literal, buscando no banco de dados por um registro cuja coluna de login fosse exatamente igual a esse texto completo.
- **Resultado Obtido:** A consulta retornou um conjunto vazio (results.length === 0), impedindo o acesso e disparando a resposta HTTP controlada com o código 401 Unauthorized e a mensagem tratada: "Usuário ou senha inválidos!". O sistema resistiu totalmente à tentativa de invasão, mantendo a integridade da aplicação.

B2.3 GERENCIAMENTO SEGURO DE CREDENCIAIS

Em conformidade com as boas práticas de desenvolvimento seguro, as credenciais de acesso ao banco de dados (como host, usuário e senha do SGBD) encontram-se rigorosamente isoladas do código-fonte principal.

No projeto atual, as configurações de conexão são lidas de variáveis de ambiente locais ou de um arquivo de configuração dedicado (config.json / .env) que não é versionado no repositório Git remoto (adicionado ao arquivo .gitignore), prevenindo a exposição acidental de senhas e dados sensíveis de infraestrutura.

B3 TRANSAÇÕES E CONTROLE DE CONCORRÊNCIA

O Sistema Gerenciador de Loja (SGL) foi projetado para garantir a atomicidade, consistência, isolamento e durabilidade (ACID) das operações críticas, especialmente em processos de vendas que envolvem múltiplas tabelas interdependentes e abatimento de estoque.

B3.1 IMPLEMENTAÇÃO DE TRANSAÇÕES NA CAMADA DE APLICAÇÃO

Operações complexas — como o cadastro de um novo pedido (POST /api/pedidos) — exigem que várias ações ocorram de forma totalmente integrada: criação do pedido, inserção

dos itens na tabela associativa, abatimento do saldo de estoque dos produtos, registro do histórico de status inicial e inclusão de pagamentos.

Se qualquer uma dessas etapas falhar (por exemplo, estoque insuficiente ou erro de sintaxe), todo o lote de operações deve ser revertido para evitar registros órfãos ou inconsistências financeiras. Para isso, o SGL utiliza o gerenciamento explícito de transações do driver do MySQL (db.beginTransaction(), db.commit() e db.rollback()).

O trecho a seguir ilustra a transação robusta implementada na rota de criação de pedidos no server.js:

```
app.post('/api/pedidos', (req, res) => {
    const { id_vendedor, id_cliente, data_emissao, pagamentos, status_inicial } = req.body;
    let { itens } = req.body;
    if (!itens || itens.length === 0) {
        return res.status(400).json({ error: 'O pedido deve conter pelo menos um item.' });
    }

    // 1. Início da transação ACID
    db.beginTransaction((errTrans) => {
        if (errTrans) return res.status(500).json({ error: 'Erro interno no servidor.' });

        let itensProcessados = [];
        let valorTotalCalculado = 0;
        let validacoesPendentes = itens.length;
        let erroProcessamento = null;

        itens.forEach(i => {
            db.query('SELECT estoque, nome, preco_unitario FROM produto WHERE id_produto = ?',
[i.id_produto], (errP, resP) => {
                if (errP || resP.length === 0) {
                    erroProcessamento = `Produto ID ${i.id_produto} não encontrado.`;
                } else {
                    const prod = resP[0];
                    if (prod.estoque < i.quantidade) {
                        // Se houver falta de estoque, a transação é abortada (Rollback)
                        erroProcessamento = `Estoque insuficiente para o produto "${prod.nome}".`;
                    } else {
                        const precoUnitarioOficial = Number(prod.preco_unitario);
                        const subtotal = Number(i.quantidade) * precoUnitarioOficial;
                        valorTotalCalculado += subtotal;
                        itensProcessados.push({
                            id_produto: i.id_produto,
                            quantidade: i.quantidade,
                            preco_unitario: precoUnitarioOficial,
                            subtotal: subtotal
                        });
                    }
                }

                validacoesPendentes--;
                if (validacoesPendentes === 0) {
                    if (erroProcessamento) {
                        return db.rollback(() => res.status(400).json({ error: erroProcessamento }));
                    }

                    // 2. Inserção do pedido principal
```

```
46                  const queryPedido = `INSERT INTO pedido (id_cliente, id_vendedor, valor_total)
    VALUES (?, ?, ?)`;
47                  db.query(queryPedido, [id_cliente, id_vendedor, valorTotalCalculado], (errPed, resultPed)
    => {
48                      if (errPed) {
49                          return db.rollback(() => res.status(500).json({ error: 'Erro ao cadastrar pedido.' }));
50                      }
51
52                      const idPedido = resultPed.insertId;
53                      const valoresItens = itensProcessados.map(ip => [idPedido, ip.id_produto,
    ip.quantidade, ip.preco_unitario, ip.subtotal]);
54
55                      // 3. Inserção dos itens e baixa de estoque de forma atômica
56                      db.query(`INSERT INTO item_pedido (id_pedido, id_produto, quantidade,
    preco_unitario, subtotal) VALUES ?`, [valoresItens], (errItens) => {
57                          if (errItens) {
58                              return db.rollback(() => res.status(500).json({ error: 'Erro ao registrar itens.' }));
59                          }
60
61                          // 4. Confirmação definitiva da transação (Commit)
62                          db.commit((errCommit) => {
63                              if (errCommit) return db.rollback(() => res.status(500).json({ error: 'Erro no
    commit.' }));
64                              res.status(201).json({ message: 'Pedido cadastrado com sucesso!', id_pedido:
    idPedido });
65                          });
66                      });
67                  });
68              }
69          });
70      });
71  });
72  });
```

B3.2 CONTROLE DE CONCORRÊNCIA E ISOLAMENTO (PREVENÇÃO DE CONDIÇÃO DE CORRIDA)

Para demonstrar o controle de concorrência no SGL, descreve-se o cenário de Condição de Corrida (Race Condition) no estoque e como a arquitetura do banco de dados e da aplicação impede anomalias como leituras sujas (dirty reads) ou vendas simultâneas acima do estoque físico.

- **Cenário de Teste Concorrente:** Dois vendedores tentam vender o último item remanescente de um produto de ID 10 (estoque atual = 1 unidade) exatamente no mesmo microssegundo.
- **Comportamento sem controle:** Sem o isolamento transacional adequado, ambas as requisições leriam estoque = 1, validariam a compra com sucesso, inseririam dois pedidos e atualizariam o estoque para -1, violando a Regra de Negócio RN10 (estoque não-negativo).
- **Solução e Mecanismo de Controle adotado no SGL:**

1. **Motor InnoDB do MySQL:** Todas as tabelas do projeto utilizam o mecanismo ENGINE=InnoDB, que fornece suporte nativo a transações e bloqueio de linhas (row-level locking) durante instruções de modificação (UPDATE / SELECT ... FOR UPDATE).
2. **Validação Atômica por Rollback:** Como demonstrado no código da API, a transação valida o estoque e executa a baixa de forma síncrona. Se a primeira transação concluir a alteração, a segunda transação que tentar ler ou modificar o mesmo registro de estoque encontrará o saldo atualizado e disparará o rollback, emitindo a mensagem de erro controlada ("Estoque insuficiente").

B4 OTIMIZAÇÃO DE CONSULTAS

Para atender ao requisito de desempenho do sistema, foram selecionadas três consultas críticas da aplicação (relacionadas a relatórios analíticos e listagens complexas). Para cada uma delas, registou-se o plano de execução original (EXPLAIN), criou-se um índice estratégico, avaliou-se o novo plano e mediu-se o ganho de desempenho.

B4.1 CONSULTA 1: RELATÓRIO DE VENDAS POR CLIENTE

- **Objetivo de Negócio:** Agrupar os pedidos e calcular o valor total gasto por cada cliente para um determinado vendedor, com suporte a junções entre cliente, pessoa e pedido.
- **Comando SQL Analisado:**

```
EXPLAIN
SELECT c.id_pessoa AS id_clientepessoa, p_pes.nome AS cliente_nome,
    COUNT(ped.id_pedido) AS total_pedidos,
    SUM(ped.valor_total) AS valor_gasto_total
FROM cliente c
JOIN pessoa p_pes ON c.id_pessoa = p_pes.id_pessoa
JOIN pedido ped ON c.id_pessoa = ped.id_cliente
WHERE ped.id_vendedor = 1
GROUP BY c.id_pessoa, p_pes.nome;
```

Análise do Plano de Execução (Antes vs. Depois)

- **Antes da Intervenção (explain_antes-1.png):** O SGBD utilizava um índice genérico de chave estrangeira (fk_pedido_vendedor), resultando num custo superior e uso de tabela temporária (Using temporary) para efetuar o agrupamento por cliente.
- **Índice Criado:**

```
CREATE INDEX idx_pedido_vendedor_cliente ON pedido(id_vendedor, id_cliente);
```

- **Justificativa do Índice:** A escolha de um índice composto com id_vendedor na primeira posição permite filtrar de imediato os registros do vendedor logado, enquanto a segunda coluna (id_cliente) acelera o relacionamento e o agrupamento subsequente.

- **Depois da Intervenção (explain_depois-1.png):** O plano de execução passou a utilizar a chave idx_pedido_vendedor_cliente, otimizando a varredura das linhas na tabela de fatos e reduzindo o esforço de indexação nas junções.

B4.2 CONSULTA 2: RELATÓRIO DE PRODUTOS VENDIDOS E RECEITA

- **Objetivo de Negócio:** Cruzar o catálogo de produtos com a tabela associativa de itens para auditar a quantidade total comercializada e a receita gerada por item.
- **Comando SQL Analisado:**

```
EXPLAIN
SELECT pr.id_produto, pr.nome AS produto_nome,
    SUM(ip.quantidade) AS qtd_total_vendida,
    SUM(ip.subtotal) AS receita_total
FROM produto pr
JOIN item_pedido ip ON pr.id_produto = ip.id_produto
WHERE pr.id_vendedor = 1
GROUP BY pr.id_produto, pr.nome;
```

Análise do Plano de Execução (Antes vs. Depois)

- **Antes da Intervenção (explain_antes-2.png):** O acesso à tabela associativa item_pedido apoiava-se apenas na restrição de chave estrangeira padrão, gerando maior custo de leitura nas junções de grande volume.
- **Índice Criado:**

```
CREATE INDEX idx_item_pedido_produto ON item_pedido(id_produto);
```

- **Justificativa do Índice:** Como as consultas de relatório agrupam massivamente por produto, indexar a coluna id_produto na tabela de itens acelera o casamento de chaves (hash/ref join) com a tabela produto.
- **Depois da Intervenção (explain_depois-2.png):** O plano passou a apontar o uso direto de idx_item_pedido_produto na coluna key, garantindo um acesso por referência (type: ref) altamente eficiente em vez de varreduras sequenciais.

B4.3 CONSULTA 3: LISTAGEM DE PEDIDOS COM SUBCONSULTA DE STATUS MAIS RECENTE

- **Objetivo de Negócio:** Listar os pedidos do vendedor associados ao nome do cliente, recuperando dinamicamente o estado mais recente a partir da tabela de histórico cronológico (historico_status_pedido).
- **Comando SQL Analisado:**

```
EXPLAIN
SELECT p.*, pes.nome AS cliente_nome,
    (SELECT h.status FROM historico_status_pedido h WHERE h.id_pedido = p.id_pedido ORDER BY
h.data_alteracao DESC LIMIT 1) AS status_atual
FROM pedido p
```

```
JOIN cliente c ON p.id_cliente = c.id_pessoa
JOIN pessoa pes ON c.id_pessoa = pes.id_pessoa
WHERE p.id_vendedor = 1;
```

Análise do Plano de Execução (Antes vs. Depois)

- **Antes da Intervenção (explain_antes-3.png):** A subconsulta correlacionada dependia da chave primária composta básica da tabela fraca, obrigando o motor do banco a custos adicionais de ordenação (Using filesort) para determinar o registro mais recente via LIMIT 1.
- **Índice Criado:**

```
CREATE INDEX idx_historico_pedido_data ON historico_status_pedido(id_pedido, data_alteracao DESC);
```

- **Justificativa do Índice:** Como a subconsulta filtra por id_pedido e ordena estritamente por data_alteracao DESC, um índice ordenado descendentemente elimina o esforço de filesort em memória, entregando o topo da pilha de forma imediata.
- **Depois da Intervenção (explain_depois-3.png):** Observa-se que o otimizador passou a utilizar o índice idx_historico_pedido_data na subconsulta, agilizando a recuperação do status e otimizando o desempenho geral da rota de listagem de pedidos.

B4.4 DISCUSSÃO HONESTA E CONCLUSÃO DOS TESTES

Nota técnica: Em ambientes de testes com volumetria reduzida (massas iniciais de desenvolvimento), a diferença de milissegundos entre o uso e a ausência de índices pode parecer marginal. Contudo, os planos de execução (EXPLAIN) comprovam que o comportamento lógico do SGBD evoluiu de buscas genéricas ou completas para o acesso estruturado por árvores B-Tree (ref), garantindo a escalabilidade da aplicação para cenários de produção com milhares de registos transacionais.

B5 SEGURANÇA E CONTROLE DE ACESSO

A camada de segurança e controle de acesso do Sistema Gerenciador de Loja (SGL) foi estruturada sob o princípio do menor privilégio (Principle of Least Privilege - POLP), garantindo o isolamento multiusuário no banco de dados MySQL, o encapsulamento do acesso a dados sensíveis por meio de visões e o tratamento transparente da gestão de credenciais e dados pessoais.

B5.1 DEFINIÇÃO E CRIAÇÃO DE PERFIS DE ACESSO (GRANT / REVOKE)

Para assegurar que cada tipo de ator interaja com o banco de dados dentro de limites estritamente operacionais, foram modelados e criados três perfis de acesso distintos no SGBD:

1. **sgl_admin (Administrador do Sistema / DBA):** Possui privilégios totais de DDL e DML sobre o esquema sgl_db. É responsável pela manutenção de tabelas, índices e gestão global de usuários.
2. **sgl_vendedor (Operador Comercial / Aplicação Backend):** Perfil de execução operacional utilizado pelo servidor Node.js. Possui permissões de leitura, inserção e atualização nas tabelas de negócios (produtos, clientes, pedidos, itens, histórico e pagamentos), mas não possui privilégios de exclusão física permanente (DELETE) sobre dados históricos, nem privilégios de DDL (DROP, ALTER).
3. **sgl_auditor (Perfil de Leitura e Relatórios):** Perfil de acesso restrito (apenas SELECT), utilizado para extração de relatórios analíticos e auditorias. Não possui permissão para modificar qualquer dado nem visualizar colunas contendo senhas criptografadas.

Comandos SQL de Criação e Concessão de Privilégios:

```
-- 1. CRIAÇÃO DOS ROLES / PERFIS
CREATE ROLE IF NOT EXISTS 'sgl_admin';
CREATE ROLE IF NOT EXISTS 'sgl_vendedor';
CREATE ROLE IF NOT EXISTS 'sgl_auditor';

-- 2. ATRIBUIÇÃO DE PRIVILÉGIOS PARA O PERFIL ADMINISTRADOR
GRANT ALL PRIVILEGES ON sgl_db.* TO 'sgl_admin';

-- 3. ATRIBUIÇÃO DE PRIVILÉGIOS PARA O PERFIL VENDEDOR (Princípio do Menor Privilégio)
GRANT SELECT, INSERT, UPDATE ON sgl_db.pessoa TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE ON sgl_db.cliente TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE ON sgl_db.vendedor TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE ON sgl_db.categoria TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE ON sgl_db.produto TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE, DELETE ON sgl_db.fornecedor TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE, DELETE ON sgl_db.fornecedor_produto TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE, DELETE ON sgl_db.pedido TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE, DELETE ON sgl_db.item_pedido TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE, DELETE ON sgl_db.historico_status_pedido TO 'sgl_vendedor';
GRANT SELECT, INSERT, UPDATE, DELETE ON sgl_db.pagamento TO 'sgl_vendedor';
GRANT SELECT ON sgl_db.vw_relatorio_consolidado TO 'sgl_vendedor';

-- Revoga explicitamente permissão de alteração estrutural no banco para o vendedor
REVOKE DROP, ALTER, CREATE ON sgl_db.* FROM 'sgl_vendedor';

-- 4. ATRIBUIÇÃO DE PRIVILÉGIOS PARA O PERFIL AUDITOR (Apenas Leitura)
GRANT SELECT ON sgl_db.vw_relatorio_consolidado TO 'sgl_auditor';
GRANT SELECT ON sgl_db.produto TO 'sgl_auditor';
GRANT SELECT ON sgl_db.categoria TO 'sgl_auditor';
GRANT SELECT ON sgl_db.historico_status_pedido TO 'sgl_auditor';
-- Bloqueia expressamente qualquer consulta direta do auditor às credenciais
REVOKE SELECT ON sgl_db.vendedor FROM 'sgl_auditor';

-- 5. CRIAÇÃO DOS USUÁRIOS E ATRIBUIÇÃO DOS ROLES
CREATE USER IF NOT EXISTS 'user_admin'@'localhost' IDENTIFIED BY 'AdminPass123!';
CREATE USER IF NOT EXISTS 'user_app'@'localhost' IDENTIFIED BY 'AppServerPass456!';
CREATE USER IF NOT EXISTS 'user_auditor'@'localhost' IDENTIFIED BY 'AuditorPass789!';
```

```
GRANT 'sgl_admin' TO 'user_admin'@'localhost';
GRANT 'sgl_vendedor' TO 'user_app'@'localhost';
GRANT 'sgl_auditor' TO 'user_auditor'@'localhost';

-- Definir roles ativas por padrão no login
SET DEFAULT ROLE 'sgl_admin' TO 'user_admin'@'localhost';
SET DEFAULT ROLE 'sgl_vendedor' TO 'user_app'@'localhost';
SET DEFAULT ROLE 'sgl_auditor' TO 'user_auditor'@'localhost';

FLUSH PRIVILEGES;
```

B5.2 RESTRIÇÃO DE ACESSO A DADOS VIA VISÃO (VIEW)

Para impedir a exposição direta de tabelas transacionais contendo informações sensíveis do cliente (como o CPF) e assegurar a restrição de colunas e linhas para usuários do nível de consulta, utiliza-se no projeto a visão vw_relatorio_consolidado.

A visão realiza a junção entre pedido, cliente e pessoa, aplicando a regra de isolamento do vendedor e encapsulando a subconsulta de recuperação do status mais recente do pedido.

```
CREATE OR REPLACE VIEW vw_relatorio_consolidado AS
SELECT
    p.id_pedido,
    p.id_vendedor,
    p.data_emissao,
    p.valor_total,
    c.id_pessoa AS id_cliente,
    pes_cliente.nome AS cliente_nome,
    pes_cliente.cpf AS cliente_cpf,
    (SELECT h.status
     FROM historico_status_pedido h
     WHERE h.id_pedido = p.id_pedido
     ORDER BY h.data_alteracao DESC
     LIMIT 1) AS status_atual
FROM pedido p
JOIN cliente c ON p.id_cliente = c.id_pessoa AND p.id_vendedor = c.id_vendedor
JOIN pessoa pes_cliente ON c.id_pessoa = pes_cliente.id_pessoa;
```

Mecanismo de Proteção e Isolamento:

A aplicação restringe a consulta da visão parametrizando o filtro por id_vendedor, garantindo que um operador nunca acesse registros consolidation pertencentes a outros vendedores:

```
1   // Consulta na API Node.js utilizando a VIEW com restrição por vendedor
2   app.get('/api/relatorios/consolidado', (req, res) => {
3   const { id_vendedor } = req.query;
4   if (!id_vendedor) {
5   return res.status(400).json({ error: 'ID do vendedor não fornecido.' });
6   }
7
8   const query = 'SELECT * FROM vw_relatorio_consolidado WHERE id_vendedor = ?';
9   db.query(query, [id_vendedor], (err, results) => {
10  if (err) return res.status(500).json({ error: 'Erro ao gerar relatório consolidado' });
11  res.json(results);
12  });
13  });
```

B5.3 TRATAMENTO E ARMAZENAMENTO DE SENHAS (HASH)

Em conformidade com as diretrizes do projeto, as credenciais dos usuários com perfil vendedor (vendedor.senha_hash) foram modeladas para armazenar hashes criptográficos com suporte a até 255 caracteres.

Esclarecimento sobre a Simulação Utilizada:

No protótipo funcional atual da aplicação Node.js (server.js), o armazenamento e a verificação de senhas encontram-se temporariamente implementados com strings simuladas no formato hash_ficticio_8f31a2. O grupo assume conscientemente essa simplificação acadêmica para exercitar a camada de persistência em banco de dados, reconhecendo a penalização eventual de pontuação neste item.

Implementação Recomendada/Alvo (BCrypt com Salt):

Em ambiente de produção real, o tratamento de senhas na camada de aplicação deve utilzar o algoritmo BCrypt com fator de custo (salt) igual ou superior a 10:

```
1   // Padrão de produção para hashing seguro de senha com Salt
2   const bcrypt = require('bcrypt');
3   const saltRounds = 10;
4
5   // Exemplo de Hash na Criação de Conta
6   const senhaPlana = req.body.senha;
7   const senhaHash = await bcrypt.hash(senhaPlana, saltRounds);
8
9   // Exemplo de Validação no Login
10  const match = await bcrypt.compare(senhaPlana, vendedor.senha_hash);
```

B5.4 TRATAMENTO DE DADOS PESSOAIS E LGPD

Conforme a Lei Geral de Proteção de Dados Pessoais (LGPD - Lei nº 13.709/2018) e as normas de ética da disciplina, o projeto adotou diretrizes rigorosas para o tratamento de informações sensíveis:

B5.4.1 Mapeamento de Dados Pessoais Identificados:

Na base de dados sgl_db, os seguintes atributos das tabelas pessoa, vendedor, cliente e fornecedor constituem Dados Pessoais e Dados Pessoais Sensíveis:

- cpf e cnpj: Identificadores únicos nacionais.
- nome, email, telefone: Dados de qualificação e contato direto.
- login e senha_hash: Credenciais privadas de autenticação.

B5.4.2 Medidas de Proteção e Boas Práticas Adotadas:

- **Uso Exclusivo de Dados Fictícios:** A base de carga (02_carga.sql) e os testes de interface utilizam dados sintéticos e fictícios. É expressamente proibido o uso de CPFs ou e-mails reais de terceiros.
- **Minimização do Acesso:** As consultas parametrizadas limitam os campos retornados às telas do sistema (por exemplo, omitindo a coluna senha_hash nos retornos da API de login e das rotas de relatórios).
- **Inativação Lógica (RN27):** Atendendo ao princípio do direito do titular e preservando a integridade histórica fiscal, o descarte de registros não realiza a exclusão física (DELETE CASCADE) de pessoas com histórico transacional, mas sim a inativação lógica através da flag ativo = FALSE.

B6 GATILHOS E AUDITORIA

Para assegurar a rastreabilidade estrita de alterações críticas no Sistema Gerenciador de Loja (SGL), foram implementados dois gatilhos (triggers) no SGBD MySQL. Os gatilhos monitoram operações sensíveis do ecossistema comercial e financeiro: a alteração de preços de produtos e a modificação de status de pedidos.

B6.1 ESTRUTURA DA TABELA DE AUDITORIA

Para centralizar e registrar de forma perene os eventos auditados, criou-se a tabela auditoria_sistema. Esta tabela armazena a identificação do registro afetado, a tabela de origem, o tipo de operação realizada (UPDATE), os valores armazenados antes e depois da modificação, a data da ocorrência e o identificador do operador responsável (id_vendedor).

Script DDL da Tabela de Auditoria:

```
-- Tabela centralizadora de auditoria do sistema
CREATE TABLE IF NOT EXISTS auditoria_sistema (
    id_auditoria INT AUTO_INCREMENT PRIMARY KEY,
    tabela_afetada VARCHAR(50) NOT NULL,
    id_registro INT NOT NULL,
    operacao VARCHAR(20) NOT NULL,          -- Ex: 'UPDATE_PRECO', 'UPDATE_STATUS'
    id_vendedor INT NULL,                   -- Operador responsável pela alteração
    valor_antigo TEXT NULL,                 -- Estado dos dados antes da modificação
    valor_novo TEXT NULL,                   -- Estado dos dados após a modificação
    data_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

B6.2 IMPLEMENTAÇÃO DO GATILHO 1: AUDITORIA DE ALTERAÇÃO DE PREÇOS DE PRODUTOS

- **Justificativa do Domínio:** O preço de venda de um produto (produto.preco_unitario) é um dado financeiro sensível. Alterações indevidas ou não autorizadas podem gerar prejuízos comerciais ou conflitos com clientes. A auditoria garante a conformidade e o histórico de flutuação de preços.
- **Tabela Monitorada:** produto
- **Momento de Disparo:** BEFORE UPDATE (permitindo capturar o estado original antes de sobrescrevê-lo)

Código SQL do Gatilho 1:

```
DELIMITER //

CREATE TRIGGER trg_auditoria_preco_produto
BEFORE UPDATE ON produto
FOR EACH ROW
BEGIN
   -- Verifica se houve alteração efetiva no preço unitário
   IF OLD.preco_unitario <> NEW.preco_unitario THEN
      INSERT INTO auditoria_sistema (
         tabela_afetada,
         id_registro,
         operacao,
         id_vendedor,
         valor_antigo,
         valor_novo
      )
      VALUES (
         'produto',
         OLD.id_produto,
         'UPDATE_PRECO',
         NEW.id_vendedor,
         CONCAT('Preço Anterior: R$ ', OLD.preco_unitario),
         CONCAT('Novo Preço: R$ ', NEW.preco_unitario)
      );
   END IF;
END//

DELIMITER ;
```

B6.3 IMPLEMENTAÇÃO DO GATILHO 2: AUDITORIA DE ALTERAÇÃO DE STATUS DE PEDIDOS

- **Justificativa do Domínio:** O ciclo de vida do pedido (transições entre PENDENTE, PAGO, FINALIZADO e CANCELADO) rege o fluxo financeiro e logístico da loja. Rastrear quem e quando modificou o estado de um pedido assegura governança contra fraudes ou reversões indevidas de vendas.

- **Tabela Monitorada:** historico_status_pedido (ou diretamente na tabela pedido caso utilize uma coluna de status centralizada. No SGL, as mudanças de status registram-se via inserção no histórico; logo, o gatilho monitora a evolução de estados críticos).
- **Momento de Disparo:** AFTER INSERT na tabela de histórico de status do pedido.

Código SQL do Gatilho 2:

```
DELIMITER //

CREATE TRIGGER trg_auditoria_status_pedido
AFTER INSERT ON historico_status_pedido
FOR EACH ROW
BEGIN
   DECLARE v_id_vendedor INT;

   -- Recupera o id_vendedor associado ao pedido em questão
   SELECT id_vendedor INTO v_id_vendedor
   FROM pedido
   WHERE id_pedido = NEW.id_pedido;

   -- Insere o registro na tabela de auditoria central
   INSERT INTO auditoria_sistema (
      tabela_afetada,
      id_registro,
      operacao,
      id_vendedor,
      valor_antigo,
      valor_novo
   )
   VALUES (
      'pedido',
      NEW.id_pedido,
      'MUDANCA_STATUS',
      v_id_vendedor,
      'Status Anterior (Ver historico_status_pedido)',
      CONCAT('Novo Status: ', NEW.status)
   );
END//

DELIMITER ;
```

B6.4 VALIDAÇÃO E EVIDÊNCIA DE FUNCIONAMENTO

Durante a demonstração ao vivo para o professor (conforme o roteiro da Etapa 2), ao realizar uma alteração de preço em um produto pela interface ou via comando direto no banco:

1. O gatilho trg_auditoria_preco_produto é disparado automaticamente de forma transparente pelo SGBD InnoDB.
2. Um comando simples de consulta na tabela de auditoria revelará o evento gravado:

```
SELECT * FROM auditoria_sistema ORDER BY data_hora DESC;
```

3. O resultado exibirá o autor (id_vendedor), a data/hora exata, e os valores formatados contendo o valor anterior e o valor posterior, satisfazendo integralmente o requisito B6 da avaliação.

B7 PROJETO FÍSICO E ARMAZENAMENTO

B7.1 REVISÃO DOS TIPOS DE DADOS À LUZ DO DOMÍNIO REAL

Durante o desenvolvimento e a consolidação do esquema físico (01_ddl.sql), as escolhas dos tipos de dados foram reavaliadas considerando o comportamento e o custo de armazenamento no SGBD MySQL (InnoDB).

- **Identificadores (INT vs BIGINT):**
  - Estado Atual: Utilizou-se INT AUTO_INCREMENT para as chaves primárias numéricas (ex: id_produto, id_pedido, id_pessoa).
  - Avaliação Crítica / O que mudaria: Para o horizonte de dois anos em tabelas de alto volume (como item_pedido e historico_status_pedido), o tipo INT suporta até 2.147.483.647 registros com sinal. Caso o volume operacional ultrapasse essa marca, a alteração recomendada seria a migração para BIGINT UNSIGNED, expandindo o espaço de endereçamento e prevenindo estouros de capacidade (integer overflow), embora incorra em um leve aumento no tamanho dos índices B-Tree associados.
- **Dados Textuais Restritos (CHAR vs VARCHAR):**
  - Estado Atual: Atributos de tamanho fixo regulamentado por leis ou normas de identificação (como cpf CHAR(11) e cnpj CHAR(14)) foram mantidos como CHAR.
  - Avaliação Crítica: A escolha foi correta, pois evita a sobrecarga de bytes de comprimento (length prefix bytes) exigida pelo tipo VARCHAR e garante alocação estática previsível nas páginas de dados do InnoDB para chaves de tamanho estrito.
- **Valores Monetários e Estoque:**
  - Estado Atual: DECIMAL(10,2) para preços e subtotais, e INT para estoques e quantidades.
  - Avaliação Crítica: O tipo DECIMAL foi essencial para evitar os erros de arredondamento inerentes a números de ponto flutuante (FLOAT/DOUBLE), assegurando exatidão contábil e financeira nas transações de vendas e custos.

B7.2 ESTIMATIVA DE CRESCIMENTO DAS TABELAS DE MAIOR MOVIMENTO (HORIZONTE DE 2 ANOS)

Para dimensionar o armazenamento e planejar a infraestrutura de I/O do SGBD, projetou-se o crescimento das duas tabelas de maior transacionalidade do SGL: item_pedido e historico_status_pedido.

B7.2.1 Tabela item_pedido

- **Premissas de Dimensionamento:**
  - Média estimada: 500 pedidos finalizados por dia.
  - Média de itens por pedido: 3 itens.
  - Volume diário gerado: 1.500 linhas/dia →≈ 45.000 linhas/mês → 540.000 linhas/ano.
  - Horizonte de 2 anos: ≈ 1.080.000 registros.
- **Estimativa de Espaço Físico:**
  - Cada linha possui colunas inteiras (id_pedido, id_produto, quantidade ≈ 12 bytes) e decimais (preco_unitario, subtotal ≈ 16 bytes), além do overhead de transação e metadados do InnoDB (≈ 30 a 50 bytes por linha).
  - Tamanho estimado por linha: ≈ 80 bytes.
  - Espaço total para dados (1,08 milhão de linhas): 1.080.000 × 80 bytes ≈ 86,4 MB.
  - Com os índices associados (PRIMARY KEY (id_pedido, id_produto) e idx_item_pedido_produto), o espaço total estimado para a tabela em 2 anos não ultrapassará 150 MB, sendo perfeitamente viável em qualquer hardware padrão.

B7.2.2 Tabela historico_status_pedido

- **Premissas de Dimensionamento:**
  - Considerre-se uma média de 4 mudanças de status por pedido (ex: PENDENTE → PAGO → FINALIZADO).
  - Para 500 pedidos/dia → 2.000 registros de histórico por dia → 60.000/mês → 720.000/ano.
  - Horizonte de 2 anos: ≈ 1.440.000 registros.
- **Estimativa de Espaço Físico:**
  - Colunas: id_pedido (4 bytes), data_alteracao (5 bytes DATETIME), status (VARCHAR(20) ≈ 10 bytes).

- Tamanho médio da linha com overhead do InnoDB: ≈ 60 bytes.
- Espaço total para dados (1,44 milhão de linhas): $1.440.000 \times 60$ bytes ≈ 86,4 MB.
- Com índices, o consumo projetado para 2 anos é de aproximadamente 140 MB.

B7.3 PLANO DE BACKUP, RETENÇÃO E RECUPERAÇÃO

Garantir a durabilidade e a integridade dos dados comerciais exige uma estratégia formal de salvaguarda.

- **O que copiar (Escopo do Backup):**
  - **Backup Lógico Completo:** Arquivos de dump estrutural e de dados gerados via mysqldump contendo todo o esquema do banco sgl_db, tabelas transacionais, visões e procedimentos.
  - **Logs de Transação (Binlogs):** Habilitação do Binary Logging (log-bin) no MySQL para permitir recuperações pontuais (Point-in-Time Recovery - PITR).
- **Frequência e Horário:**
  - **Backup Completo (Full Backup):** Diariamente, executado de madrugada (ex: às 02:00 AM), período de menor atividade operacional da loja.
  - **Backup Incremental / Binlog:** Salvamento contínuo das alterações transacionais ao longo do dia útil.
- **Estratégia de Retenção:**
  - **Diários:** Manter os backups dos últimos 7 dias.
  - **Semanais:** Manter um backup por semana durante o período de 1 mês.
  - **Mensais:** Arquivamento de um backup por mês com retenção legal e fiscal de 5 anos (atendendo a exigências fiscais brasileiras sobre faturamento e emissão de notas/pedidos).
- **Procedimento de Teste de Restauração:**
  - Mensalmente, a equipe de administração do banco deve simular uma restauração em ambiente de homologação (container Docker isolado) para validar a integridade dos arquivos:

```
# Exemplo de comando de restauração validado
mysql -u root -p sgl_db < backup_sgl_db_2026-10-27.sql
```

- Após a importação, executa-se uma consulta de conferência de totais na visão consolidada (SELECT COUNT(*) FROM vw_relatorio_consolidado;) para atestar que nenhum dado foi corrompido.

B7.4 PROPOSTA DE PARTICIONAMENTO E ARQUIVAMENTO HISTÓRICO

Dado que o volume projetado para dois anos é relativamente modesto ($\approx 1{,}5$ milhão de registros nas tabelas maiores), o motor InnoDB do MySQL gerencia essa volumetria com alto desempenho utilizando apenas os índices B-Tree dimensionados no artefato B4.

No entanto, projetando um cenário de alta escala corporativa (expansão para dezenas de lojas e milhões de pedidos anuais), recomenda-se a seguinte estratégia de particionamento por intervalo (Range Partitioning):

- **Tabela Alvo:** historico_status_pedido e pedido.
- **Critério de Particionamento:** Particionamento anual ou mensal baseado na coluna de data (data_emissao ou data_alteracao).
- **Benefício Operacional:** Permite realizar operações de arquivamento por rotação de partições (Partition Purging/Swapping). Partições antigas (com mais de 2 anos, por exemplo) podem ser desanexadas rapidamente (ALTER TABLE ... DROP PARTITION) sem a necessidade de varreduras custosas linha a linha via comandos DELETE, eliminando o bloqueio de tabelas (table locking) e o crescimento descontrolado do arquivo de índice primário.